

1. Sign up for a free AWS account (if you don't have one), log in to the AWS Management Console, and navigate to the AWS Lambda service dashboard.

ANS:

- I signed up for a free AWS account and logged in to the **AWS Management Console**.

- ❖ Steps followed:

1. Open the AWS Management Console.
 2. Search for **Lambda** in the AWS services search bar.
 3. Open the **AWS Lambda** service.
 4. Click **Create function**.
 5. Select **Author from scratch**.
 6. Enter the function name:
 - InstaLikeCounter
 7. Select **Python** as the runtime.
 8. Keep the default execution role settings.
 9. Click **Create function**.
2. Create a new AWS Lambda function named InstaLikeCounter that takes an event with a 'likes' number and returns the value increased by 1, simulating an Instagram-like button click.

ANS:

- I used the following Python code in the AWS Lambda code editor:

```
def lambda_handler(event, context):  
  
    likes = event.get("likes", 0)  
  
    return {  
  
        "statusCode": 200,  
  
        "likes": likes + 1  
  
    }
```

- The function receives a likes value from the event and increases it by **1**.

- ❖ For example:

- Input: 99
- Output: 100

3. Test your InstaLikeCounter Lambda function using the built-in AWS Lambda test feature by providing a sample event with 'likes': 99, and verify the output shows 100.

ANS :

- I created a test event using the following JSON:

```
{  
  "likes": 99  
}
```

- I saved the test event and clicked **Test**.

- ❖ Expected output:

```
{  
  "statusCode": 200,  
  "likes": 100  
}
```

- This confirms that the Lambda function successfully increased the likes count from **99 to 100**.

4. Write a short comparison (3-4 lines each) of AWS Lambda, Azure Functions, and Google Cloud Functions, focusing on how each lets you run code without managing servers.

ANS :

❖ **AWS Lambda**

- AWS Lambda is a serverless compute service provided by Amazon Web Services.

It runs code in response to events without requiring the user to manage servers.

It automatically handles infrastructure and can scale based on incoming requests.

❖ **Azure Functions**

- Azure Functions is Microsoft's serverless compute service.

It allows developers to run small pieces of code without managing virtual machines or servers.

It can be triggered by HTTP requests, timers, queues, and other Azure services.

❖ **Google Cloud Functions**

- Google Cloud Functions is Google's serverless execution platform for event-driven applications.

- Developers can deploy individual functions without managing the underlying servers.
- The platform automatically handles infrastructure and scales the function according to demand.

Feature	AWS Lambda	Azure Functions	Google Cloud Functions
Cloud Provider	AWS	Microsoft Azure	Google Cloud
Server Management	Not required	Not required	Not required
Execution	Event-driven	Event-driven	Event-driven
Scaling	Automatic	Automatic	Automatic

5. Use the AWS Lambda code editor to add a console.log statement that prints 'Function executed!' every time your function runs.

Hint: Check the Monitor tab after running your test to see the log output.

ANS :

- I updated the Lambda function to print "**Function executed!**" every time it runs.

```
def lambda_handler(event, context):
```

```
    print("Function executed!")
```

```
    likes = event.get("likes", 0)
```

```
    return {
```

```
        "statusCode": 200,
```

```
        "likes": likes + 1
```

```
    "likes": 99
```

```
}
```

The output was:

```
{
```

```
    "statusCode": 200,
```

```
    "likes": 100
```

```
}
```

- I then opened the **Monitor** tab and selected **View CloudWatch logs** to verify the log message.
- ❖ Expected log output:
- This confirms that the Lambda function executed successfully and generated the expected CloudWatch log.